

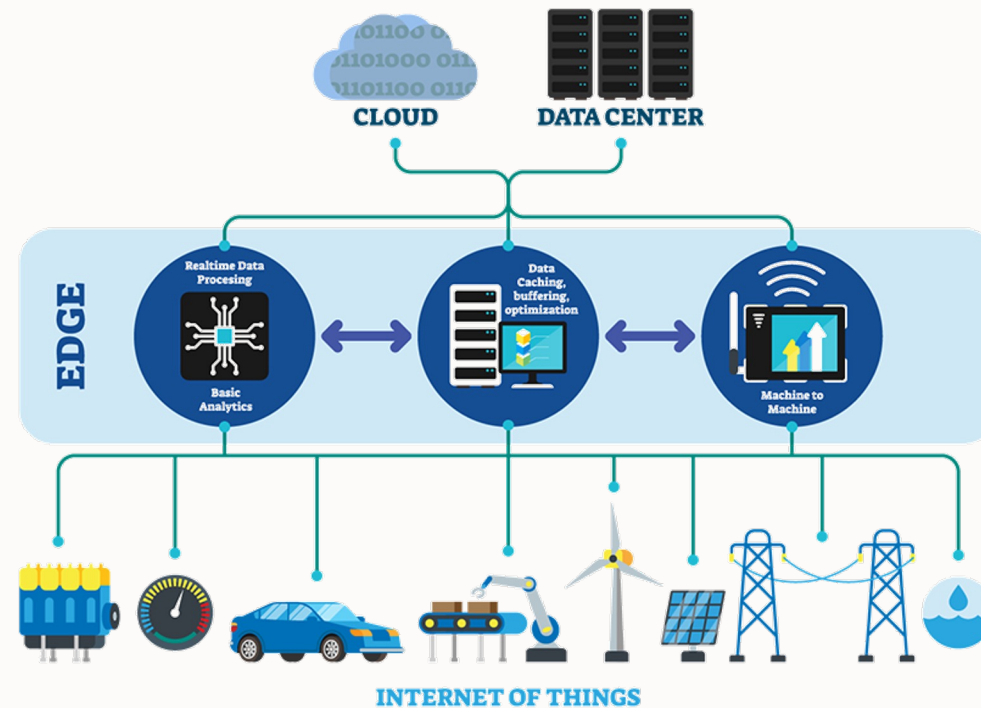


Introduction aux IoT Edges

Durée 3h

Edge Computing (1)

Edge Computing

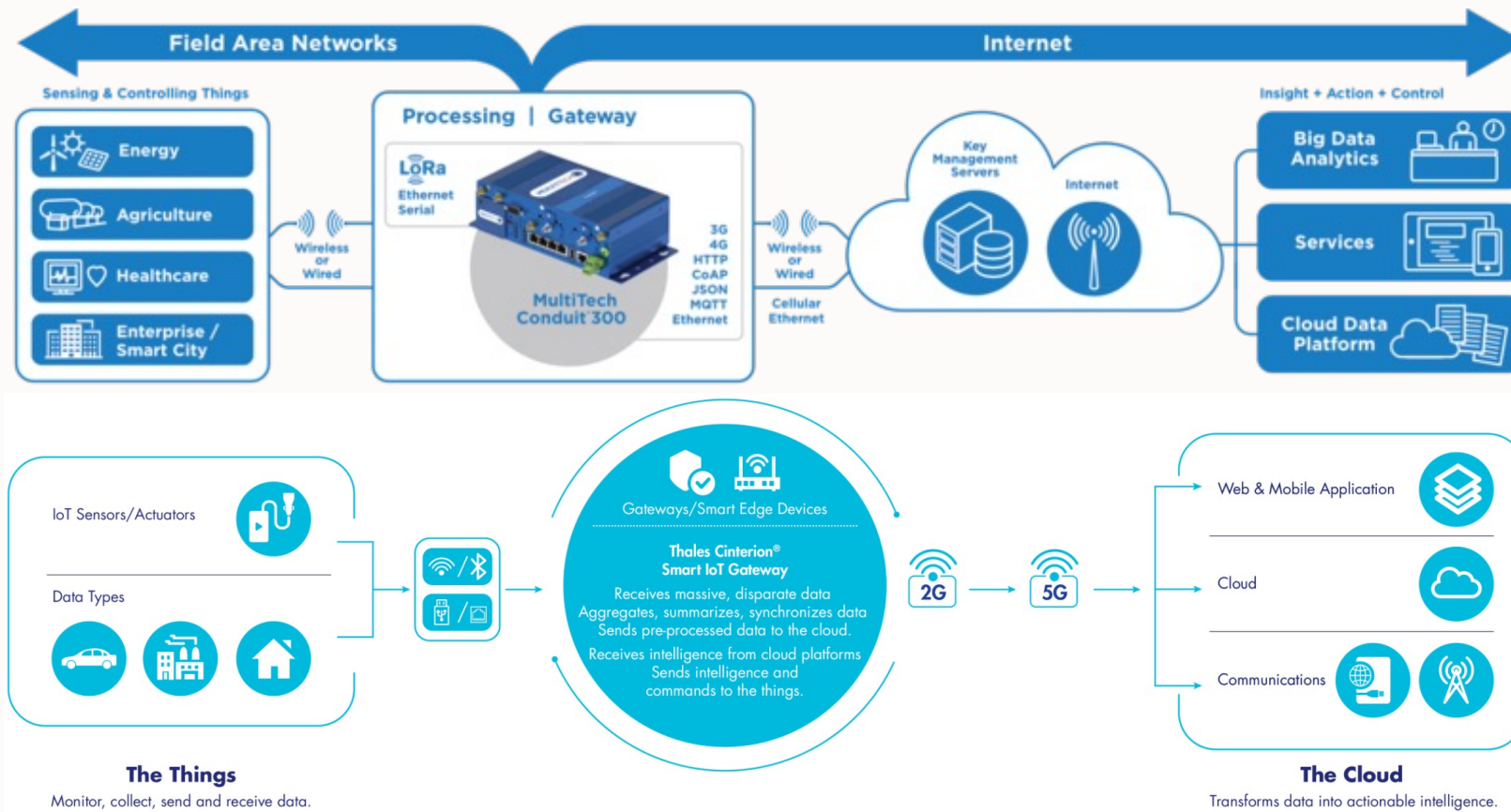


Source:

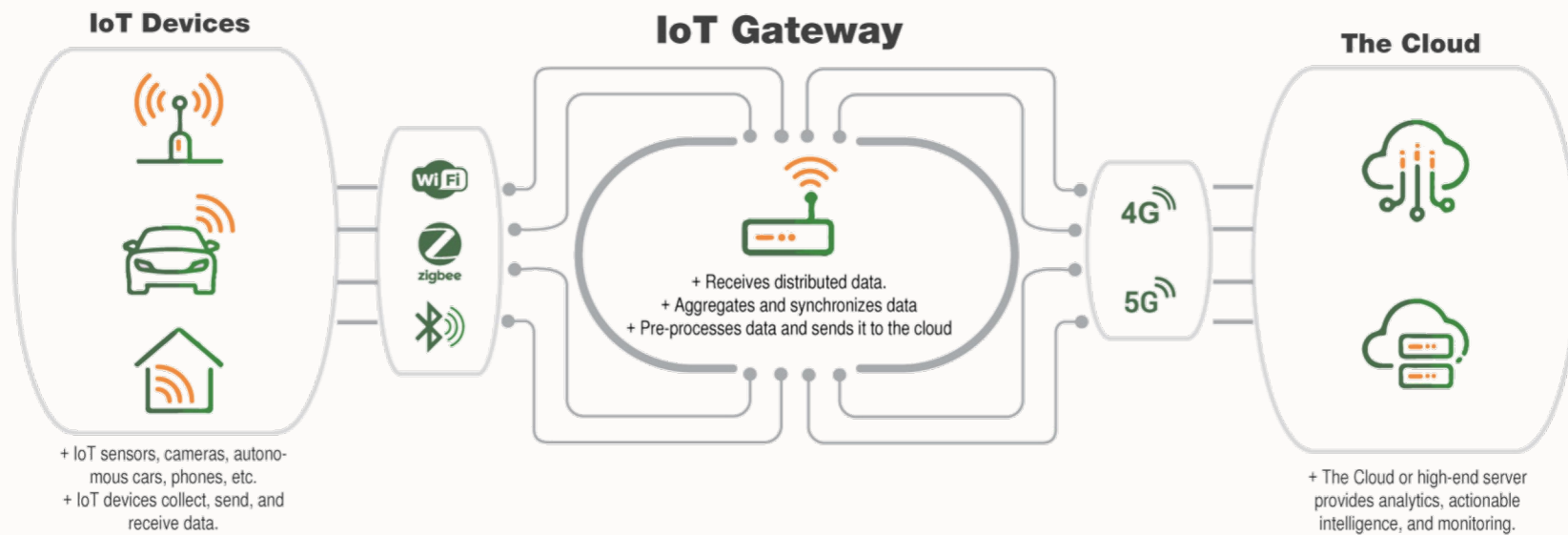
IEEE Innovation
at work



Edge Computing (2)

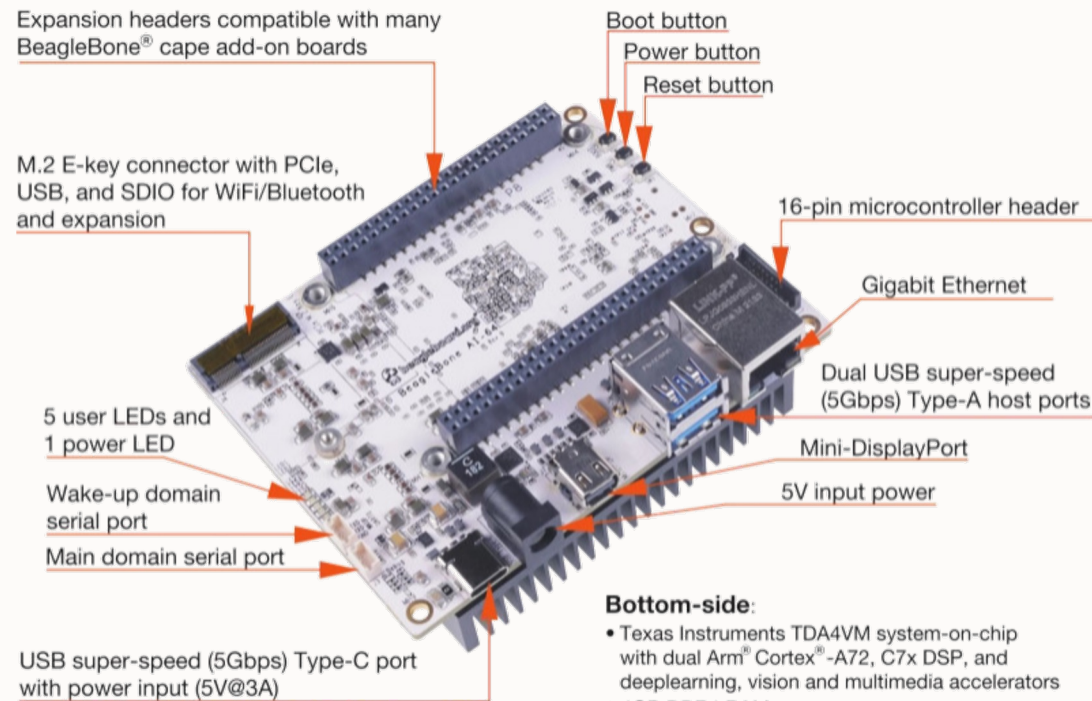


Edge Computing (3)



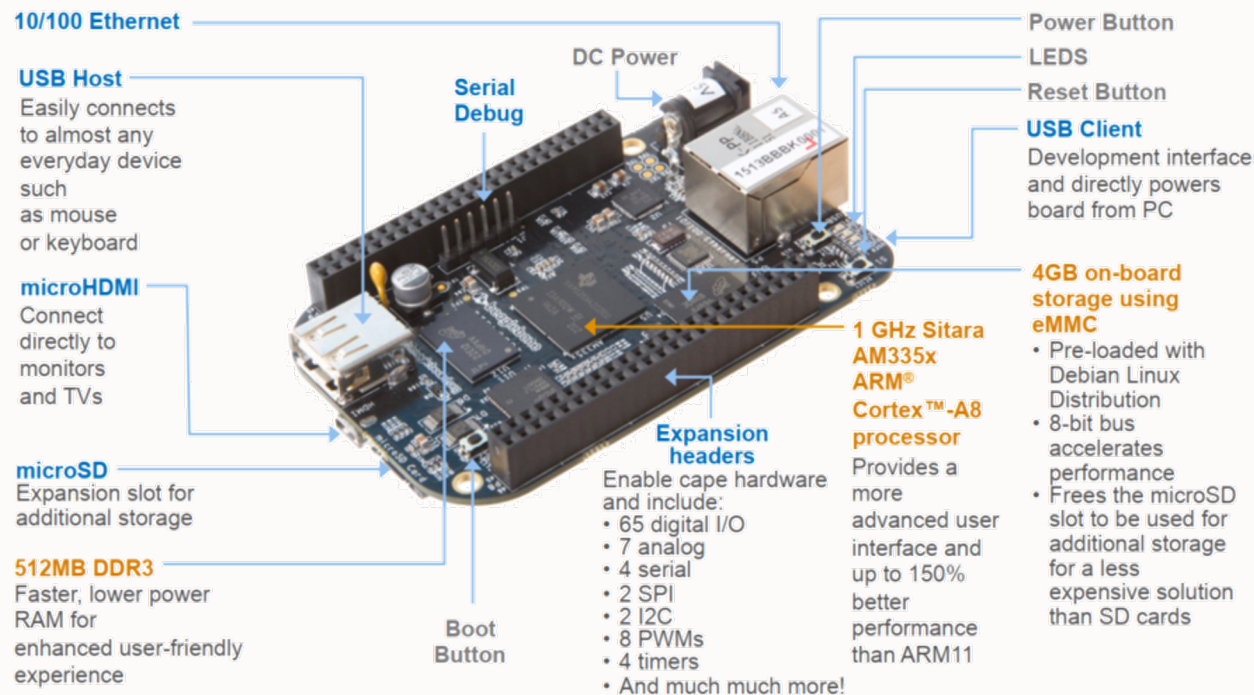
IoT Gateway (1)

BeagleBone[®] AI-64



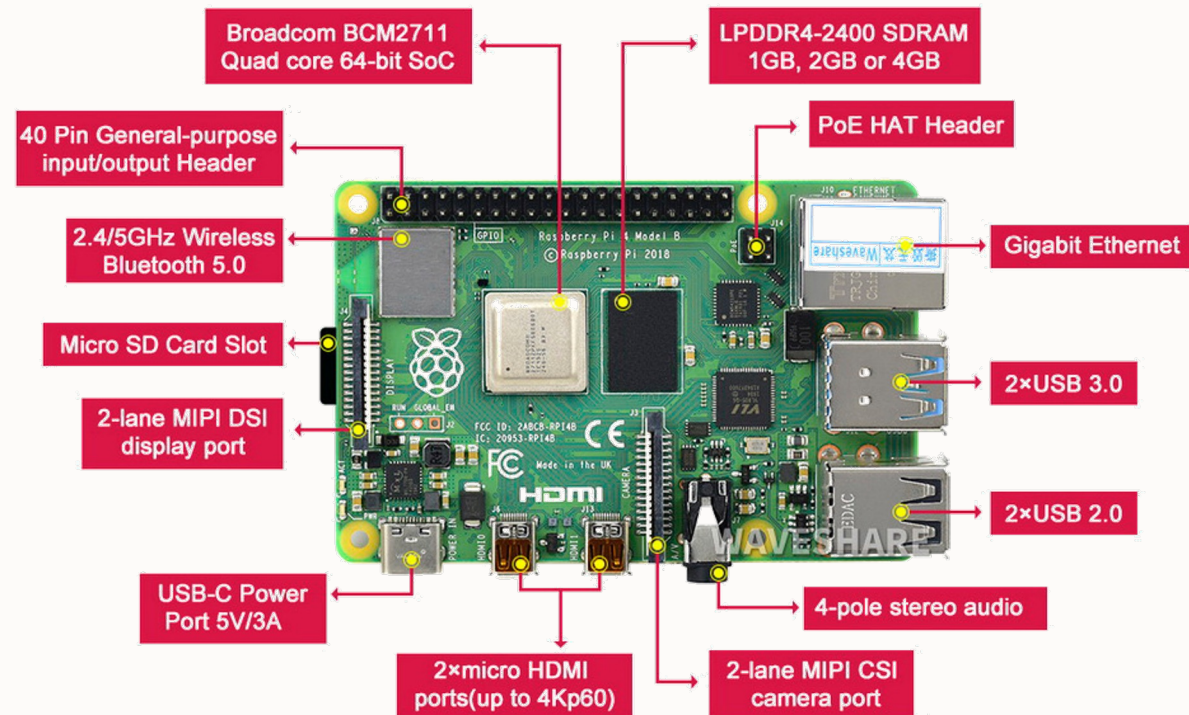
IoT Gateway (2)

Beagle Bone Black



IoT Gateway (3)

Raspberry Pi 4 Model B



IoT Gateway (4)

Raspberry Pi 400



Quad-Core Cortex-A72
64-bit SoC@1.8GHz



Always Stay Cool
With Huge Heatsink



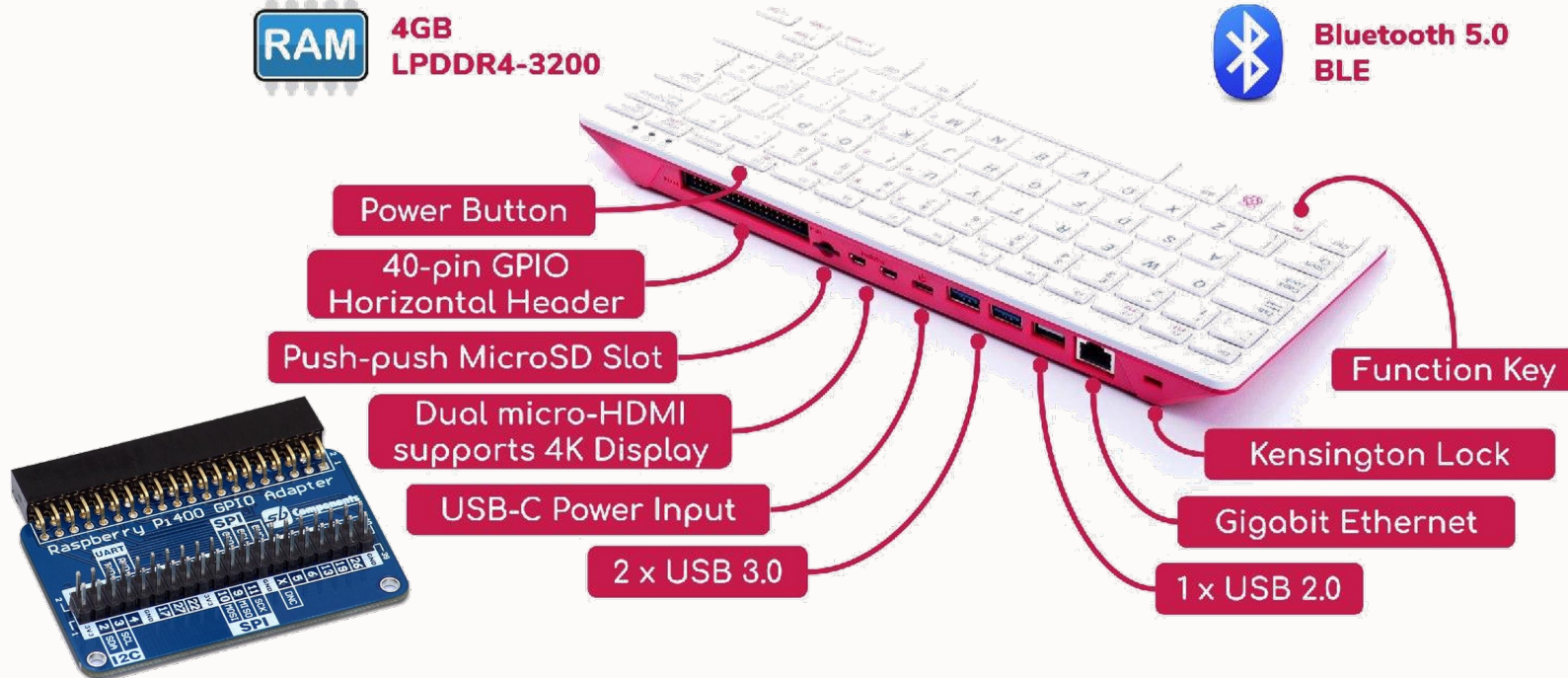
Dual-band WiFi
2.4GHz and 5GHz



4GB
LPDDR4-3200

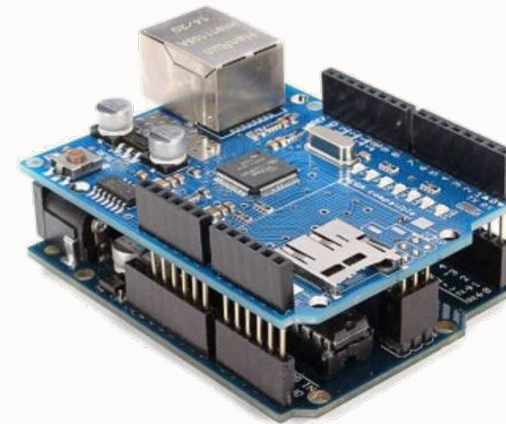
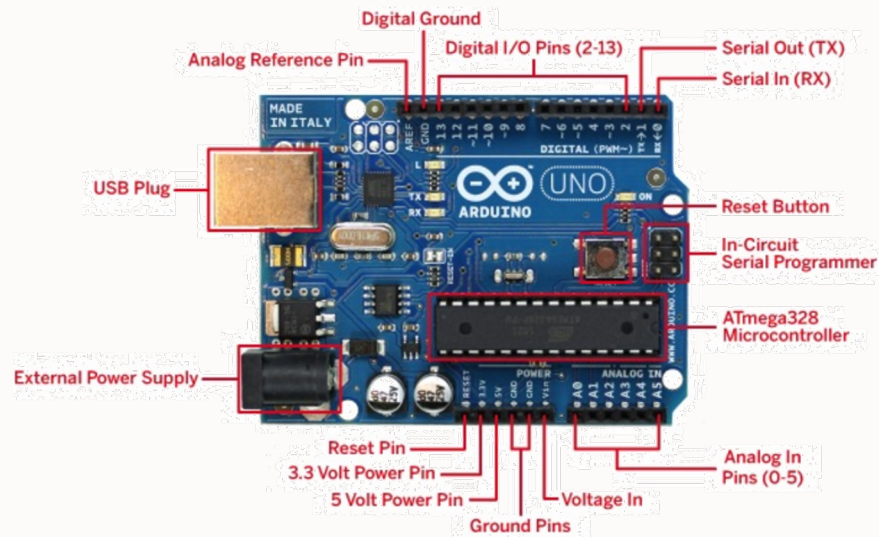


Bluetooth 5.0
BLE



IoT Gateway (5)

Arduino Uno



IoT Stack for IoT Edge (Exemple)

- Installer le langage Go : <https://go.dev> (armv6l)
- Installer Flogo CLI : <https://tibcosoftware.github.io/flogo/getting-started/getting-started-cli/>
- Installer Flogo Web UI : <https://tibcosoftware.github.io/flogo/getting-started/getting-started-webui/>
- Configurer votre application Flogo sur votre IoT Gateway :
 - IoT App : <https://tibcosoftware.github.io/flogo/labs/iot-howto/>
 - BeagleBone : <https://tibcosoftware.github.io/flogo/labs/beaglebone-iot/>
 - Raspberry Pi : <https://tibcosoftware.github.io/flogo/labs/raspberry-iot/>
- Faire communiquer Flogo avec un Broker MQTT :
 - https://docs.tibco.com/pub/flogo-mqtt/1.0.0/doc/pdf/TIB_flogo-mqtt_1.0_users_guide.pdf?id=1
- Tutoriels Flogo :
 - <https://thenewstack.io/tutorial-building-an-iot-app-with-flogo-and-raspberry-pi/>
 - <https://community.tibco.com/s/article/Machine-Learning-with-TIBCO-Connected-Intelligence>

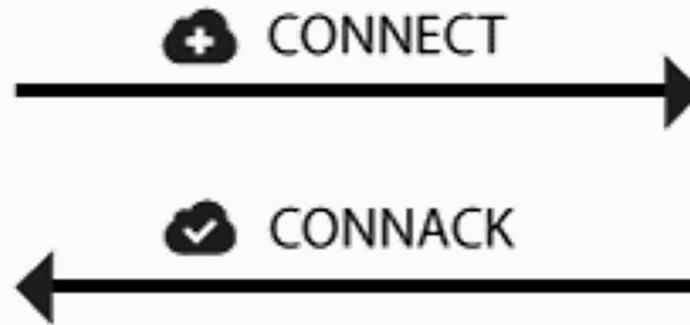
MQTT (1)

- ✓ MQTT (« Publish/Subscribe »)
 - ✓ Protocole simple et léger sur TCP
 - ✓ Créé en 1999 pour la télémétrie
 - ✓ Gratuit depuis 2010
 - ✓ Standard OASIS depuis le 29 octobre 2014
 - ✓ Brokers MQTT : HiveMQ, Eclipse Mosquitto, ActiveMQ, RabbitMQ, Apollo
- ✓ MQTT-SN (Sensor Networks)
 - ✓ Version M2M en s'affranchissant du protocole TCP
 - ✓ Fonctionne sur les réseaux M2M (ZigBee, bus série, UDP, ...)
 - ✓ Se connecte à un courtier MQTT via un MQTT-SN Gateway
 - ✓ Implémentation : [Really Small Message Broker](#)

MQTT (2)

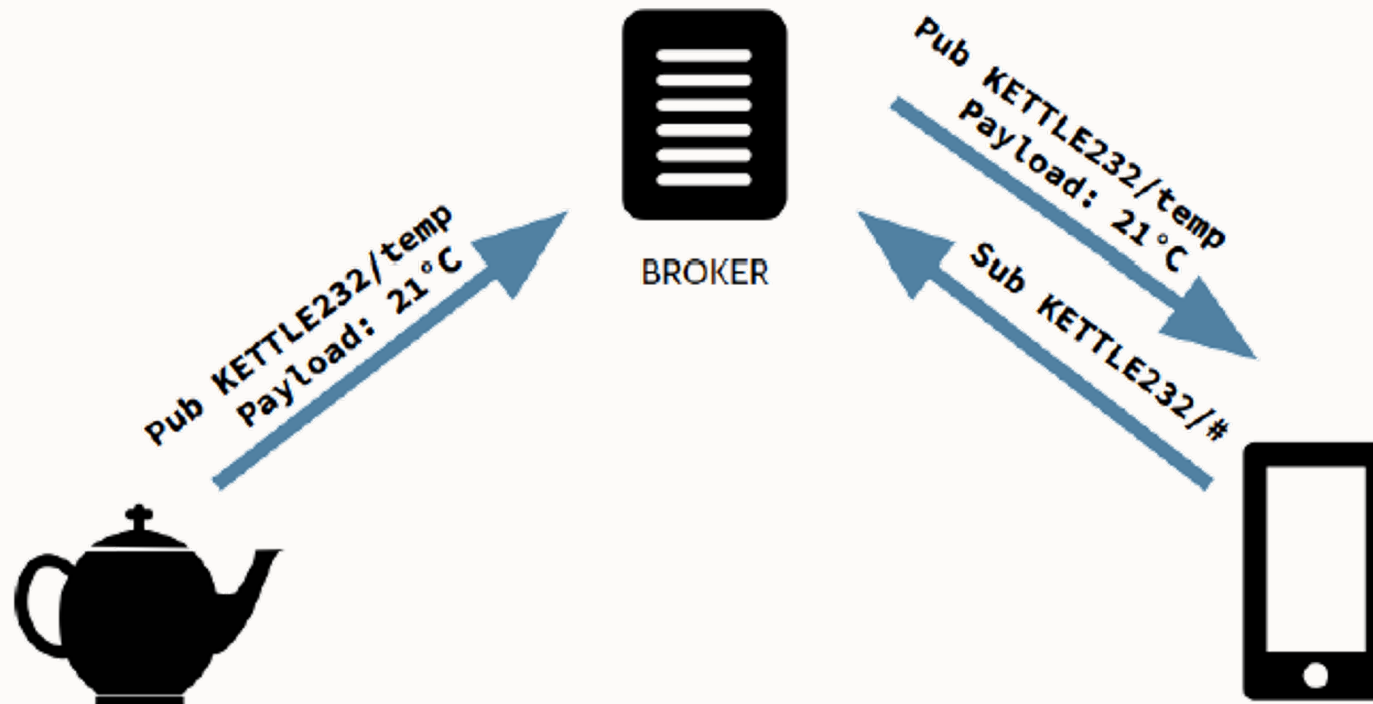


MQTT Client

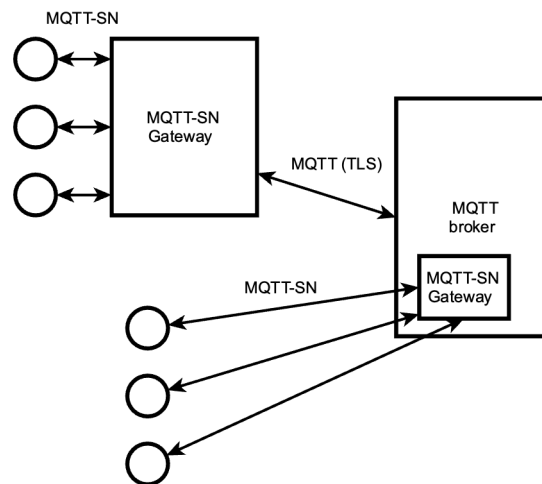
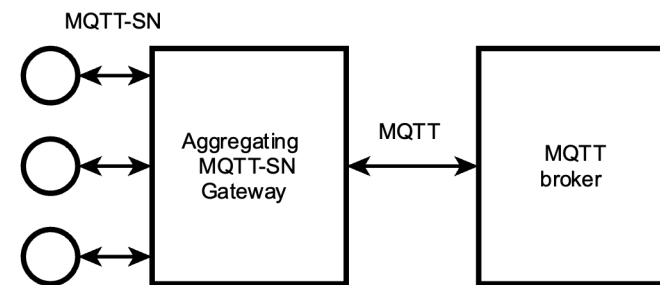
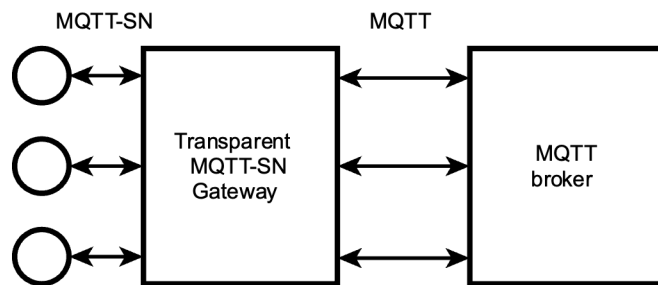


MQTT Broker

MQTT (3)



MQTT (4)

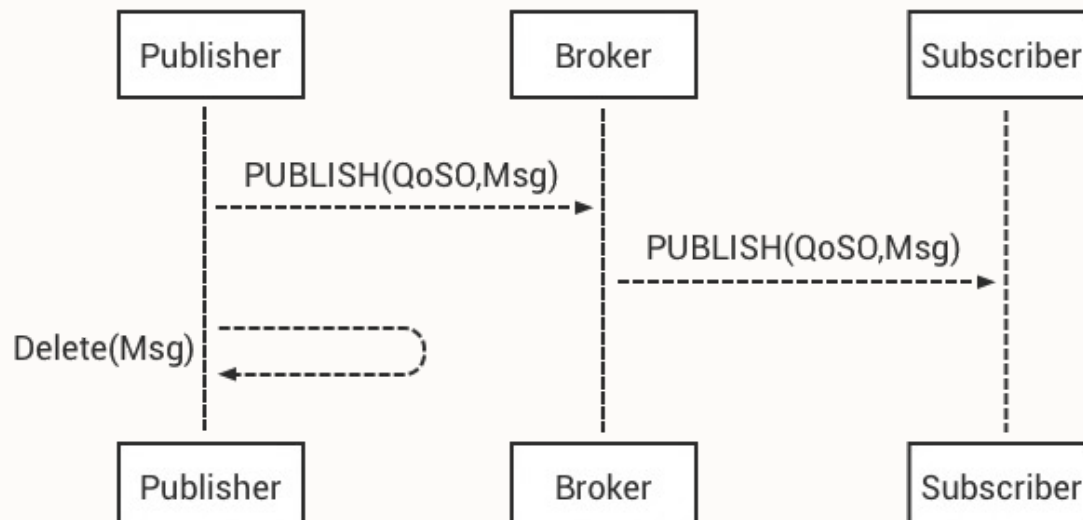


MQTT (5)

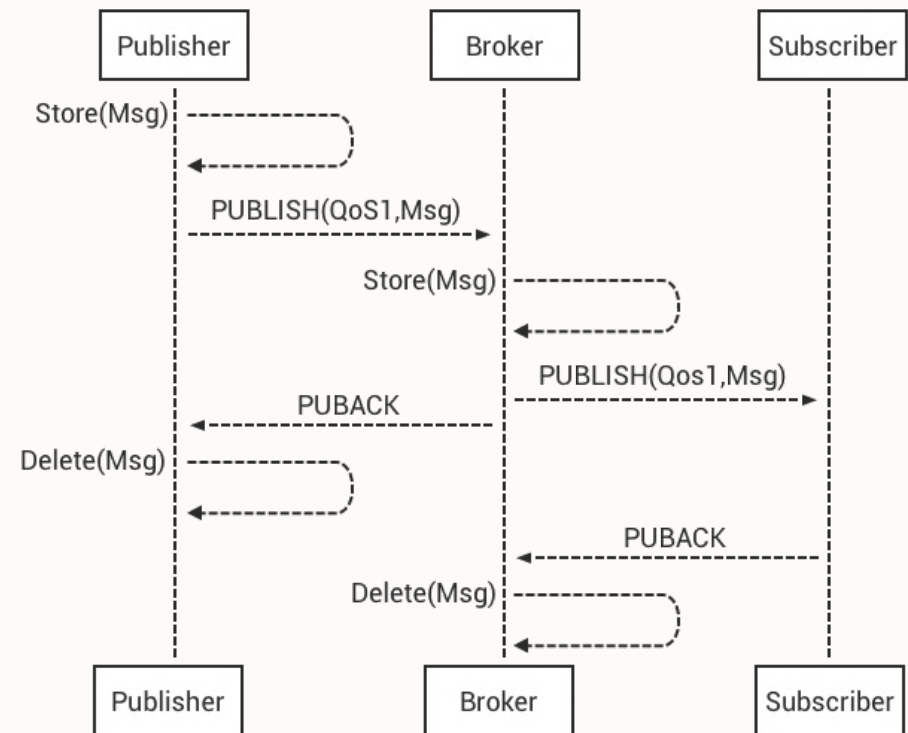
- ✓ Fonctionnalités offertes par MQTT
 - ✓ Identification capteur / émetteur de message : « Topic » (p.ex. SENSOR1TEMP, SENSOR1HUMIDITY)
 - ✓ Gestion de plusieurs capteurs et d'actuateurs :
 - ✓ hiérarchiser les « Topic » : SENSOR/1/HUMIDITY
 - ✓ Utilisation possible de deux « Topic Wildcard » :
 - ✓ '#' toute la sous-arborescence : SENSOR/# → SENSOR/1/TEMP , SENSOR/2/HUMIDITY , ...
 - ✓ '+' un seul niveau de l'arborescence : SENSOR+/TEMP → SENSOR/1/TEMP , SENSOR/2/TEMP , ...
 - ✓ Trois niveaux de qualité de services :
 - ✓ QoS0 : At most once (fire and forget) : Mode le plus rapide / Pas d'acquittement / perte possible
 - ✓ QoS1 : At least once : Peut être envoyé plusieurs fois / Attente d'acquittement (renvoi possible)
 - ✓ QoS2 : Exactly once : Mode le plus lent / Attente longue d'acquittement (renvoi possible)

MQTT (6)

QoS 0: AT most once (deliver and forget)

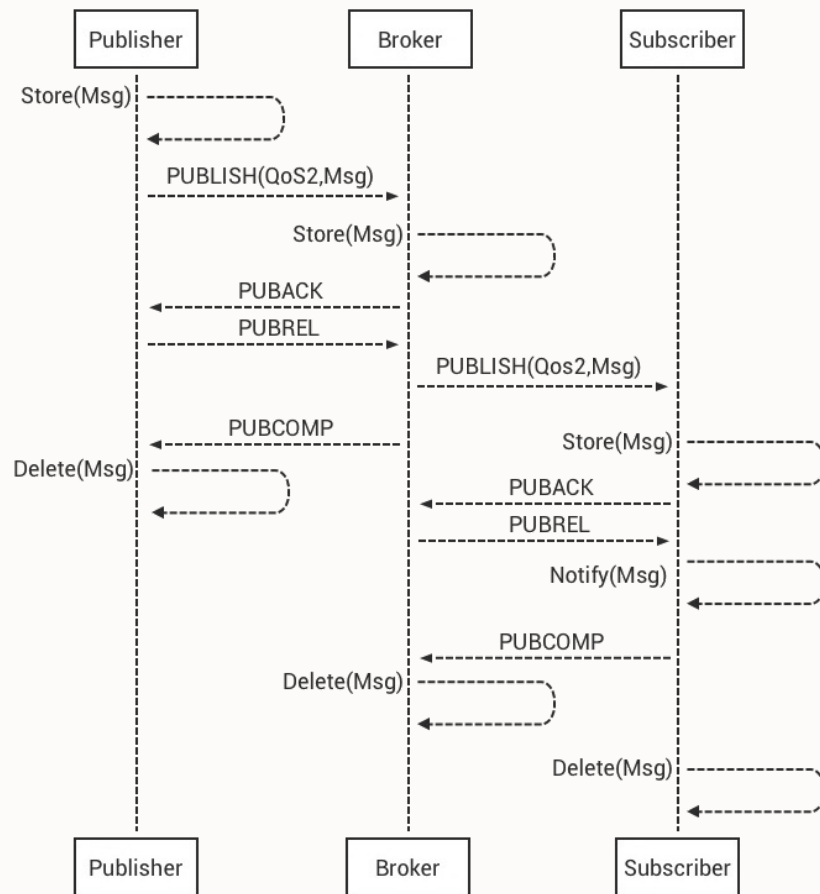


QoS 1: AT least once

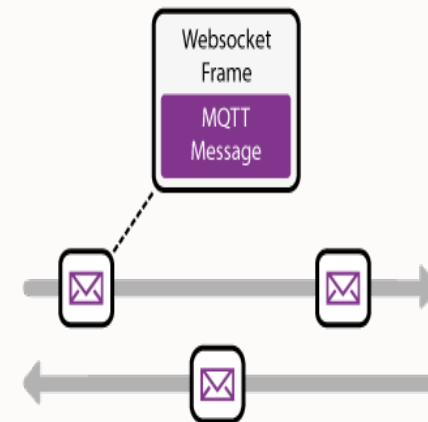


MQTT (7)

QoS 2: Exactly once



MQTT-Broker



Laptop

MQTT (8)

✓ Fonctionnalités offertes par MQTT (Suite)

- ✓ Transmission des messages en mode synchrone ou asynchrone
- ✓ « Last Will and Testament » :
 - ✓ Un capteur ou un actuateur peut se déconnecter de temps en temps (réseau non fiable, batterie épuisé,...)
 - ✓ Quand un client **se connecte** pour la première fois (CONNECT) à un MQTT broker il lui envoie son message de testament et ce dernier sauvegarde le testament.
 - ✓ Si le MQTT broker détecte que le client s'est déconnecté d'une manière inappropriée, le testament est envoyé à tous les souscripteurs à ce client.
 - ✓ Lorsque le client se déconnecte d'une manière appropriée en envoyant le message « DISCONNECT » au courtier, celui écrase le testament.
- ✓ Retained Message : un client peut sauvegarder son statut au sein du courtier. Ce statut est associé à un topic particulier (p.ex. SENSOR/1/STATUS)

MQTT (9)

- ✓ Fonctionnalités offertes par MQTT (Suite)
 - ✓ Peut être utilisé à travers les WebSockets pour communiquer en temps réel avec un client HTML 5 ou mobile
 - ✓ La sécurité peut être assurée avec le protocole TLS avec un certificat pour le broker et un mot de passe ou certificat pour le client. Une autre option
 - ✓ MQTT-SN fonctionne avec des Topic ID plutôt que des Topic Name pour réduire le volume de données véhiculant sur le réseau et par conséquent le temps de communication
- ✓ MQTT 5
 - ✓ Essentials : <https://www.hivemq.com/mqtt-5/>
 - ✓ Paho : <https://github.com/eclipse/paho.mqtt.java/blob/master/MQTTv5.md>

MQTT (10)

- ✓ Fonctionnalités offertes par MQTT (Suite)
 - ✓ C'est le MQTT-SN Gateway qui fait la correspondance entre Topic ID et Topic Name
 - ✓ En ce qui concerne la sécurité en MQTT-SN, une implémentation adéquate du Framework OAuth 2.0 est envisageable.
 - ✓ Pour plus d'information sur les fonctionnalités de MQTT consulter ces liens
 - ✓ <http://www.hivemq.com/mqtt-essentials-wrap-up/>
 - ✓ <http://www.hivemq.com/mqtt-security-fundamentals-wrap-up/>

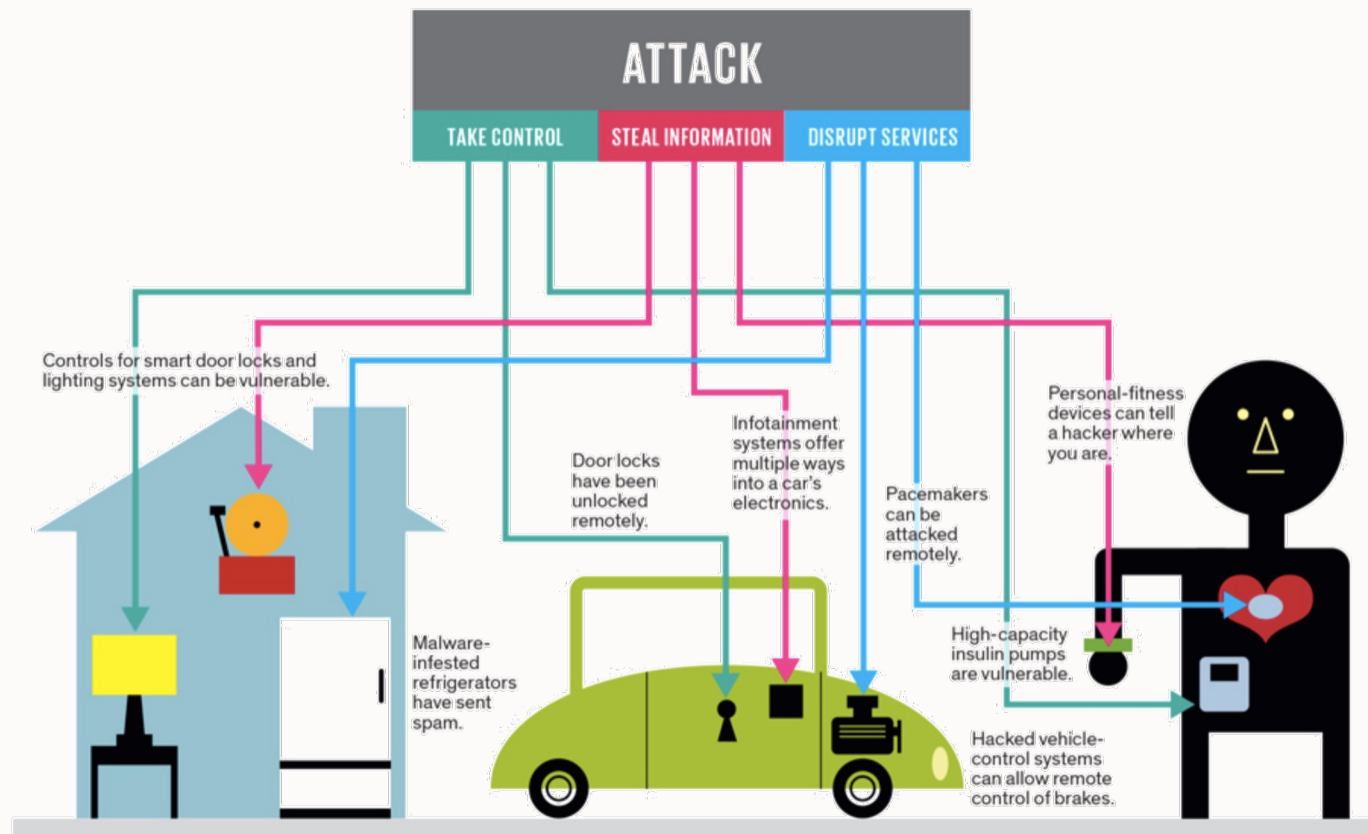
MQTT (11)

- ✓ Formats de données supportés pour les corps des messages MQTT
 - ✓ Binaires
 - ✓ Textes
 - ✓ Image encodée en base64
- ✓ Interface REST communiquant avec un courtier MQTT
 - ✓ GET → CONNECT + SUBSCRIBE
 - ✓ POST → CONNECT + PUBLISH
 - ✓ Point d'entrée pour sécuriser MQTT avec OAuth 2.0
- ✓ Interface WebSocket communiquant avec un courtier MQTT
 - ✓ Clients de messagerie / Blogs d'objets
 - ✓ Suivie en temps réel des courses / UberPop

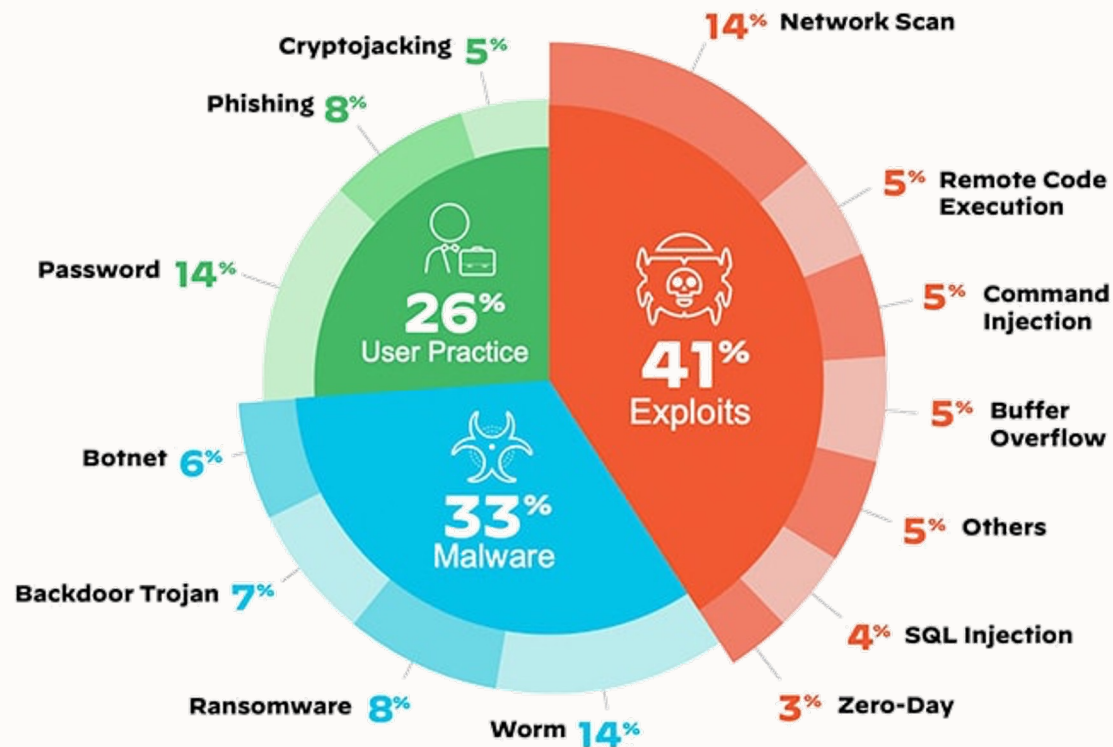
Sécurité de l'IoT Edge (1)



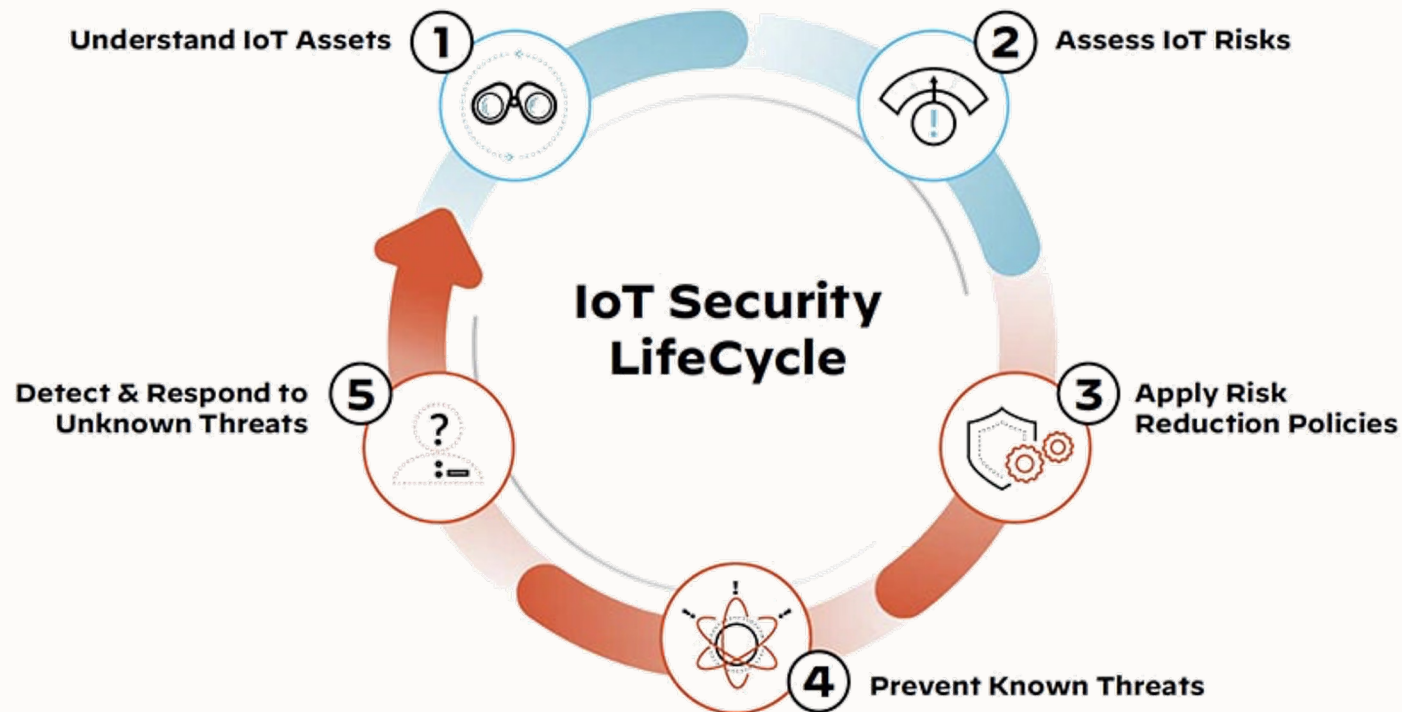
Sécurité de l'IoT Edge (2)



Sécurité de l'IoT Edge (2)



curité de l'IoT Edge (3)



Sécurité de l'IoT Edge (4)

- Définir un identifiant unique pour chaque IoT Edge
- Associer des informations sur la localisation de l'IoT Edge le cas échéant
- Associer avec l'identifiant du client, le cas échéant
- Utiliser le protocole TLS 1.2/1.3 pour se connecter au courtier (broker) MQTT:
 - Port 8883 pour se connecter à `mqtt://[username][:password]@host.domain:8883`
 - Port 8084/9001 pour se connecter à `wss://mqtt.host.domain[:port]` (MQTT over Websocket Secure)
- Limiter le Traffic par unité de temps depuis un IoT Edge vers le(s) courtier(s) MQTT pour limiter la portée des attaques de déni de service
- Utiliser un firewall avec filtrage MAC pour protéger le(s) courtier(s) MQTT et les IoT Edges